

Rayan.store

Boutique de produits informatiques & mobiles

Documentation Technique

Bachelor CDA — Concepteur Développeur d'Applications
3ème année — Promotion 2025-2026

React 18 + TypeScript

Supabase PostgreSQL

Tailwind CSS

Vite 5

Auteur : Rayan | Encadrant technique : —

Date : Mai 2026 | Version : 1.0.0

Table des matières

1. Introduction & Présentation du projet	3
1.1 Contexte et objectifs	3
1.2 Périmètre fonctionnel	3
2. Méthodologie de développement	4
2.1 Méthode agile	4
2.2 Jalons du projet	4
3. Conception UI/UX & Maquettes	5
4. Architecture base de données	6
4.1 Schéma relationnel	6
4.2 Row Level Security	7
5. Architecture applicative	8
5.1 Stack technique	8
5.2 Structure du projet	9
6. Sécurité	10
7. Tests & Qualité	11
8. Déploiement & DevOps	12
9. Conclusion & Perspectives	13

1. Introduction & Présentation du projet

1.1 Contexte et objectifs

Rayan.store est une application web e-commerce développée dans le cadre du Bachelor CDA (Concepteur Développeur d'Applications) — 3ème année. Le projet consiste à concevoir et réaliser une boutique en ligne complète spécialisée dans la vente de produits informatiques et d'appareils mobiles.

Objectif principal

Fournir une plateforme e-commerce moderne, sécurisée et performante permettant aux utilisateurs de découvrir, sélectionner et commander des produits tech avec une expérience utilisateur optimale.

Le projet répond aux 6 jalons définis dans le cahier des charges technique :

- Jalon 1** — Conception & Architecture (UML, wireframes, DB)
- Jalon 2** — Développement Front-End (React, TypeScript, Tailwind)
- Jalon 3** — Développement Back-End (Supabase, PostgreSQL, RLS)
- Jalon 4** — Intégration & Tests (Jest, Playwright)
- Jalon 5** — Sécurité & Performance (OWASP, RGPD)
- Jalon 6** — Déploiement & Documentation (CI/CD, Vercel)

1.2 Périmètre fonctionnel

Fonctionnalité	Description	Statut
Catalogue produits	Affichage, filtres par catégorie, recherche textuelle	Réalisé
Fiche produit	Détail, images, prix, stock, avis, ajout au panier	Réalisé
Panier	Ajout/suppression, quantités, persistance (local + Supabase)	Réalisé
Authentification	Inscription/connexion email, gestion session JWT	Réalisé
Profil utilisateur	Édition nom, téléphone, adresse de livraison	Réalisé

Checkout	Formulaire livraison, récapitulatif, passage commande	Réalisé
Historique commandes	Liste commandes avec statut, détail articles	Réalisé
Design responsive	Mobile (375px) → Desktop (1440px)	Réalisé

2. Méthodologie de développement

2.1 Méthode agile (Scrum adapté)

Le développement suit une méthodologie agile adaptée au contexte mono-développeur, organisée en sprints de 2 semaines :

- **Sprint 0** — Setup, architecture, wireframes (2 semaines)
- **Sprint 1** — Authentification + base de données (2 semaines)
- **Sprint 2** — Catalogue + fiche produit (2 semaines)
- **Sprint 3** — Panier + checkout (2 semaines)
- **Sprint 4** — Profil + commandes (1 semaine)
- **Sprint 5** — Tests + sécurité + déploiement (1 semaine)

2.2 Jalons et livrables

Jalon	Livrable	Date	Statut
J1 — Conception	Diagrammes UML, maquettes Figma, MCD	Semaine 2	Validé
J2 — Front-End	SPA React avec toutes les pages	Semaine 6	Validé
J3 — Back-End	Schéma Supabase + API + RLS	Semaine 6	Validé
J4 — Tests	Tests unitaires + E2E + rapport couverture	Semaine 8	En cours
J5 — Sécurité	Audit OWASP + politique RGPD	Semaine 9	Validé
J6 — Déploiement	Pipeline CI/CD + URL production	Semaine 10	Validé

3. Conception UI/UX & Maquettes

3.1 Principes de design

Le design system de Rayan.store repose sur des principes modernes visant une expérience premium :

Principe	Application
Hiérarchie visuelle	Titres Space Grotesk bold, corps Inter, gris sémantiques
Système d'espacement	Multiples de 8px (gap-2=8px, gap-4=16px, gap-8=32px...)
Palette cohérente	Primaire #2563eb, Success #10b981, Warning #f59e0b, Error #ef4444
Accessibilité	Contraste WCAG AA minimum, focus ring visible, aria-labels
Feedback utilisateur	États hover, loading spinners, toast notifications, transitions 200ms

3.2 Architecture des pages

Page	Route (état)	Description
Accueil	home	Hero, produits vedettes, catégories, promo
Catalogue	products	Grille + filtres sidebar + search
Fiche produit	product-detail	Galerie, infos, stock, ajout panier, avis
Panier	cart	Items, quantités, total, coupon
Checkout	checkout	Adresse livraison, paiement (simulé), confirmation
Authentification	auth	Login / Register avec validation
Profil	profile	Édition informations personnelles
Commandes	orders	Historique avec statut temps réel

3.3 Composants principaux

- **Header** — Navigation sticky avec scroll shadow, search overlay, dropdown catégories, menu mobile
- **ProductCard** — Image hover zoom, badge stock, prix barré, bouton ajout panier animé
- **Footer** — 4 colonnes : brand, catégories, liens, contact

4. Architecture base de données

4.1 Schéma relationnel (PostgreSQL 15 via Supabase)

La base de données comprend 8 tables métier interconnectées :

Table	Rôle	Lignes seed
categories	Catégories produits	6
products	Catalogue complet avec stock, prix, images	18
profiles	Extension auth.users (nom, tel, adresse)	—
orders	Commandes avec statut et adresse livraison	—
order_items	Lignes de commande (produit, qté, prix)	—
cart_items	Panier persistant côté serveur	—
reviews	Avis clients (note, commentaire)	—
wishlists	Favoris utilisateur	—

Modèle de données — table products

```
CREATE TABLE products (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  category_id uuid REFERENCES categories(id),  
  name        text NOT NULL,  
  slug        text UNIQUE NOT NULL,  
  description  text,  
  price       numeric(10,2) NOT NULL,  
  original_price numeric(10,2),  
  images      text[],  
  stock       integer DEFAULT 0,  
  is_featured boolean DEFAULT false,  
  specifications jsonb,  
  created_at  timestampz DEFAULT now()  
);
```

Modèle de données — table orders

```
CREATE TABLE orders (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     uuid REFERENCES auth.users(id) NOT NULL,  
  status      text DEFAULT 'pending',
```



```
total_amount      numeric(10,2) NOT NULL,  
shipping_address  jsonb,  
payment_method    text DEFAULT 'card',  
created_at        timestampz DEFAULT now(),  
updated_at        timestampz DEFAULT now()  
);
```

4.2 Row Level Security (RLS)

Principe de sécurité fondamental

Toutes les tables ont RLS activé. Aucune donnée n'est accessible sans politique explicite. Les utilisateurs ne peuvent accéder qu'à leurs propres données.

Exemples de politiques appliquées :

```
-- Lecture profil : seulement le propriétaire  
CREATE POLICY "Users can read own profile"  
  ON profiles FOR SELECT TO authenticated  
  USING (auth.uid() = id);  
  
-- Commandes : un utilisateur voit seulement les siennes  
CREATE POLICY "Users can view own orders"  
  ON orders FOR SELECT TO authenticated  
  USING (auth.uid() = user_id);  
  
-- Produits : lecture publique  
CREATE POLICY "Products are publicly readable"  
  ON products FOR SELECT TO anon, authenticated  
  USING (true);
```

5. Architecture applicative

5.1 Stack technique

Couche	Technologie	Version	Justification
UI Framework	React	18.3.1	Composants réutilisables, hooks, concurrent features
Langage	TypeScript	5.5.3	Typage fort, meilleure maintenabilité
Styling	Tailwind CSS	3.4.1	Utility-first, design system cohérent
Build tool	Vite	5.4.2	HMR ultra-rapide, ESM natif
BaaS / DB	Supabase	2.57.4	PostgreSQL managé + Auth + Realtime
Icons	Lucide React	0.344.0	Bibliothèque SVG cohérente, tree-shakeable
Fonts	Google Fonts	—	Inter (corps) + Space Grotesk (titres)

5.2 Structure du projet

```
rayan-store/
├── src/
│   ├── components/
│   │   ├── Header.tsx      # Navigation sticky responsive
│   │   ├── Footer.tsx     # Pied de page 4 colonnes
│   │   └── ProductCard.tsx # Carte produit réutilisable
│   ├── contexts/
│   │   ├── AuthContext.tsx # User, profile, signIn/Out
│   │   └── CartContext.tsx  # Panier local + Supabase sync
│   ├── lib/
│   │   └── supabase.ts      # Client singleton + types
│   ├── pages/
│   │   ├── HomePage.tsx    # Hero + featured + categories
│   │   ├── ProductsPage.tsx # Catalogue + filtres
│   │   ├── ProductDetailPage.tsx
│   │   ├── CartPage.tsx
│   │   ├── CheckoutPage.tsx
│   │   ├── AuthPage.tsx    # Login + Register
│   │   ├── ProfilePage.tsx # Édition profil
│   │   └── OrdersPage.tsx  # Historique commandes
│   ├── App.tsx             # Root + routing état
│   ├── main.tsx
│   └── index.css            # Animations + scrollbar
```

```
└─ supabase/
  └─ migrations/      # SQL migrations versionnées
  └─ public/           # Assets statiques
```

5.3 Gestion de l'état

L'application utilise le Context API natif de React avec deux contextes principaux :

Contexte	État géré	Persistance
AuthContext	user, profile, loading	Session JWT Supabase (cookie)
CartContext	items, totalItems, totalPrice	localStorage (invité) + Supabase (connecté)

Stratégie de fusion panier

Lors de la connexion, le panier localStorage est mergé avec le panier Supabase : les items existants voient leur quantité additionnée, les nouveaux items sont insérés. Le localStorage est ensuite vidé.

5.4 Routing

L'application utilise un routing basé sur l'état React (sans React Router) via le type union `Page` :

```
export type Page =
  | 'home' | 'products' | 'product-detail'
  | 'cart' | 'checkout' | 'auth'
  | 'profile' | 'orders' | 'documentation';

function navigate(page: Page, params?: {
  slug?: string;
  category?: string;
  query?: string;
}) { ... }
```

6. Sécurité

6.1 Authentication & Sessions

- **JWT Supabase** — Tokens signés RSA-256, expiration 1h, refresh automatique
- **Email/password** — Hashage bcrypt côté Supabase (non exposé client)
- **Session persistante** — Stockage `localStorage` du refresh token
- **Deadlock prevention** — Callbacks `onAuthStateChange` wrappés en IIFE async

6.2 OWASP Top 10

Risque OWASP	Mesure appliquée	Statut
A01 Broken Access Control	RLS sur toutes les tables, <code>auth.uid()</code> validé côté DB	Protégé
A02 Cryptographic Failures	HTTPS obligatoire, JWT RSA-256, bcrypt passwords	Protégé
A03 Injection	Supabase SDK paramétré (pas de SQL brut), validation TS	Protégé
A05 Security Misconfiguration	Clé anon publique (lecture seule), <code>service_role</code> côté serveur	Protégé
A07 Auth Failures	Rate limiting Supabase, erreurs generiques renvoyées	Protégé
A09 Logging Failures	Logs Supabase Dashboard + erreurs React boundary	Partiel

6.3 RGPD

Conformité RGPD

Les données personnelles (email, nom, adresse) sont stockées dans l'UE (Supabase EU region), accessibles uniquement à leur propriétaire via RLS, et peuvent être supprimées sur demande via `auth.deleteUser()`.

- Données minimales collectées (principe de minimisation)
- Consentement explicite à l'inscription (case CGV)
- Aucun cookie tracking tiers
- Politique de confidentialité accessible en footer

6.4 Variables d'environnement

```
# .env (non versionné)
VITE_SUPABASE_URL=https://xxxxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ0eS9lbnNpdjEwMDEyLWZmZWYtMTQzMTEwOTk1ODg3LnB1dC8yMDIxOGE3NDUyNTA3MC5kaWEudXMubG9naWkiLCJhdWQiOiJodHRwczovL2FkbGVldGUuc2lnbi5jb20ifQ== # Clé publique (lecture)


# .env.example (versionné)
VITE_SUPABASE_URL=your_supabase_url
VITE_SUPABASE_ANON_KEY=your_anon_key
```

7. Tests & Qualité

7.1 Stratégie de tests

La pyramide de tests adoptée pour Rayan.store :

- **Tests unitaires** (Jest + React Testing Library) — Composants, hooks, utilitaires
- **Tests d'intégration** (Jest) — Flux auth, panier, checkout
- **Tests E2E** (Playwright) — Parcours utilisateur complets

7.2 Exemples de tests unitaires

```
// CartContext – test ajout d'article
describe('CartContext', () => {
  it('should add item to cart', async () => {
    const { result } = renderHook(() => useCart(), {
      wrapper: CartProvider,
    });
    act(() => {
      result.current.addToCart({
        product_id: 'p1',
        name: 'iPhone 15 Pro',
        price: 1199,
        quantity: 1,
      });
    });
    expect(result.current.items).toHaveLength(1);
    expect(result.current.totalItems).toBe(1);
  });
});
```

7.3 Tests E2E — scénarios Playwright

Scénario	Étapes	Assertion
Parcours achat complet	Accueil → Catalogue → Produit → Panier → Checkout	Commande créée en DB
Authentification	Register → Verify → Login → Profile	Session active, profil chargé
Recherche produit	Saisie query → Résultats → Filtre catégorie	Produits filtrés correctement
Panier invité → connecté	Ajouter produit → S'inscrire → Vérifier panier	Items mergés correctement

7.4 Qualité du code

Outil	Configuration	Règles clés
TypeScript	strict: true	noImplicitAny, strictNullChecks
ESLint	eslint-plugin-react-hooks	exhaustive-deps, rules-of-hooks
Vite build	—	Chunking, tree-shaking, minification

8. Déploiement & DevOps

8.1 Pipeline CI/CD (GitHub Actions)

```
name: CI/CD Rayan.store
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run typecheck
      - run: npm run lint
      - run: npm test -- --coverage
      - run: npm run build

  deploy-preview:
    needs: build-and-test
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.VERCEL_ORG_ID }
          vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
```

8.2 Infrastructure de déploiement

Composant	Service	Configuration
Frontend	Vercel	Edge Network, CDN mondial, HTTPS auto
Base de données	Supabase Cloud	PostgreSQL 15, région EU-West-1
Auth	Supabase Auth	JWT, email/password, sessions
Storage	Supabase Storage	Images produits (futur)

8.3 Containerisation Docker

```
# Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.4 Monitoring & Performance

- **Core Web Vitals** — LCP < 2.5s, FID < 100ms, CLS < 0.1
- **Lighthouse score** — Performance 95+, Accessibilité 92+, SEO 90+
- **Bundle size** — < 300KB gzipped (code splitting par route)
- **Supabase Dashboard** — Métriques DB, logs erreurs, alertes

9. Conclusion & Perspectives

9.1 Bilan du projet

Rayan.store est une application e-commerce complète répondant aux exigences du Bachelor CDA. L'ensemble des fonctionnalités définies dans le cahier des charges a été implémenté :

Résultats atteints

8 pages fonctionnelles, 8 tables DB avec RLS, authentification sécurisée, panier persistant avec merge on login, checkout complet, historique commandes avec statuts, design responsive mobile-first.

9.2 Compétences mobilisées (référentiel RNCP)

Compétence	Mise en œuvre
CP1 — Développer une interface utilisateur	React 18, TypeScript, Tailwind, responsive
CP2 — Développer des composants d'accès aux données	Supabase SDK, requêtes SQL optimisées
CP3 — Concevoir une base de données	MCD, 8 tables, RLS, index, triggers
CP4 — Développer la partie back-end	Supabase Functions (Edge Functions)
CP5 — Mettre en place une solution de déploiement	CI/CD GitHub Actions + Vercel
CP6 — Sécuriser une application web	OWASP Top 10, RLS, JWT, RGPD

9.3 Axes d'amélioration

- Intégration d'un vrai prestataire de paiement (Stripe)
- Système de recommandations produits (IA)
- Application mobile React Native (code partagé via monorepo)
- Système de notifications push (commande expédiée, etc.)
- Optimisation SEO avec SSR (migration vers Next.js)
- Analytics e-commerce (Google Analytics 4 / Plausible)

9.4 Glossaire

Terme	Définition
RLS	Row Level Security — Sécurité au niveau des lignes PostgreSQL
JWT	JSON Web Token — Standard d'authentification sans état
BaaS	Backend as a Service — Backend managé (Supabase)
SPA	Single Page Application — Application web mono-page
SSR	Server-Side Rendering — Rendu côté serveur
CI/CD	Continuous Integration / Continuous Deployment
OWASP	Open Web Application Security Project
RGPD	Règlement Général sur la Protection des Données